Charlie Vuong
CSCI551 - Final Parallel Project
Spring 2026
Dr.Siewert

The Butterfly Effect of the Lorenz System:

Formerly “The Butterfly Effect of the Lorenz Equation”

Table of Contents:

Pg 1. Program introduction
Pg 2. Parallel design and solution with computation time
Pg 3. Parallel speed-up analysis

Program Introduction

Goal: To effectively simulate the Lorenz System and analyze the speed-up of the simulation between sequential and parallel execution on the program.

Value of solution: The Lorenz System is primarily used for modeling chaotic systems with unpredictable behavior (ie atmospheric science, fluid dynamics, etc.). Due to this, the performance requirements to simulate these behaviors are very high. Utilizing parallel programming allows massive increases in speed, thus allowing more accurate calculations and predictions.
TLDR: when the weather app tells us it is going to rain, it is less likely to lie to us.

Numerical Method(s) used: RK4

Sequential solution:
In my code, we calculate 50 trajectories, and the simulated trajectories are done one by one, sequentially.

```c
// Simulate Lorenz Trajectory
void simulate_trajectory(int trajectory_id, Point *points) {
    double x = 1.0 + CHAOS * trajectory_id; // CHAOS is our value to show the chaos over time
    double y = 1.0;
    double z = 1.0;

    // Track points for current trajectory
    int save_index = 0;

    // Time evolution loop
    for (int step = 0; step < NUM_STEPS; step++) {
        // Advance system using RK4
        rk4_step(&x, &y, &z);

        // Save point based on desired save timing
        if (step % SAVE_EVERY == 0) {
            // Compute global index for current trajectory
            int index = trajectory_id * POINTS_PER_TRAJECTORY + save_index;

            // Store current state in array
            points[index].x = x;
            points[index].y = y;
            points[index].z = z;

            // Onto the next save slot for current trajectory
            save_index++;
        }
    }
}
```

Parallel design and solution with computation time

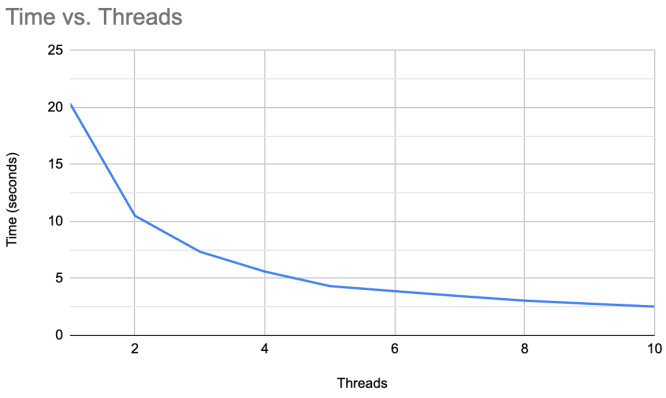
Parallel programming method(s) used: OpenMP

Parallel solution: Simulated trajectories are done in parallel. The simulated trajectories are now distributed among the threads.

Time measurement for parallel solution:

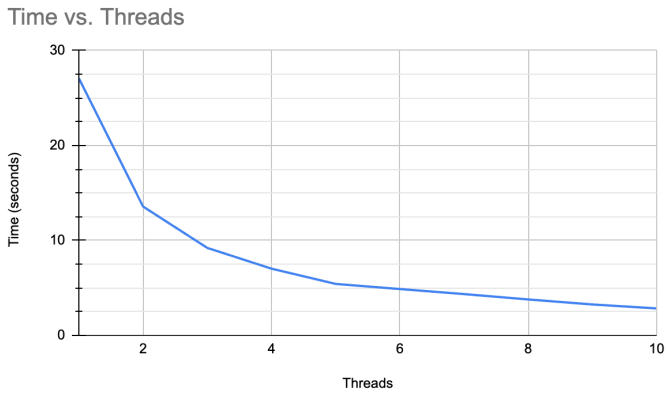
Personal system (Mac ARM)

Threads	Time (seconds)
1	20.360398
2	10.499632
3	7.344765
4	5.601659
5	4.32706
6	3.882279
7	3.450873
8	3.048965
9	2.780692
10	2.531211



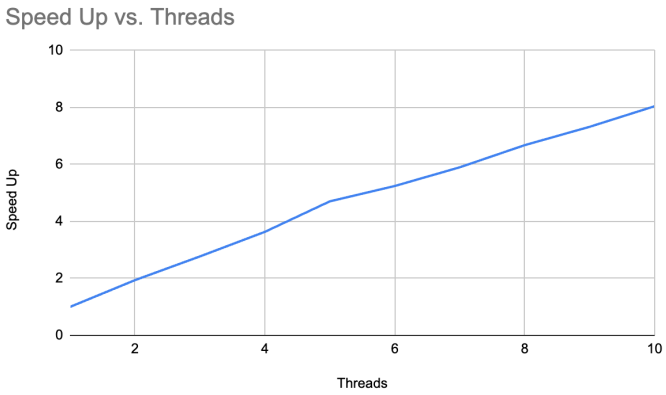
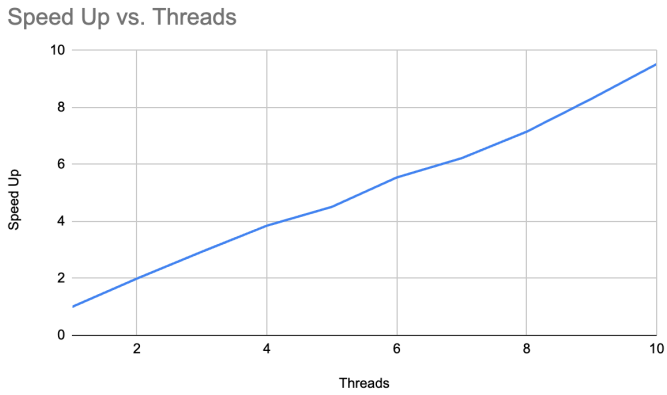
CSCI GPU Cluster

Threads	Time (seconds)
1	27.06803
2	13.567033
3	9.21428
4	7.027962
5	5.422183
6	4.882293
7	4.350744
8	3.785179
9	3.257787
10	2.842545



Parallel speed-up analysis

Speed-up Analysis and Parallel Efficiency:

Personal system (Mac ARM)	CSCI GPU Cluster
	

	Personal system (Mac ARM)	CSCI GPU Cluster
Speed Up	8.043737958x ≈ 8.04x	9.522463145x ≈ 9.52x
Parallel	0.9729774328 ≈ 97.3%	0.994427949 ≈ 99.44%
Sequential	0.02702257 ≈ 2.7%	0.00557 ≈ 0.5572%

Best scale-up and/or scale-out and offload results:

The best result I was able to obtain is running all the available threads on the CSCI GPU Cluster.

```
Lorenz Equation OpenMP Simulation
Trajectories: 50
DT: 0.000010
Steps per trajectory: 10000000
Total saved points: 50000000
Max OpenMP threads available: 16
Number of threads using: 16
Simulation time: 2.169463 seconds
```

As we can see, the benefits begin to plateau. Note that this is primarily the case with our parameters. Increasing the number of trajectories would allow for the delay of the plateau as the problem size scaling increases.

Max speed-up at 16 threads: 27.06803 / 2.169463 ≈ 12.5x